LECTURE 09

Object Arrays, Composition & Class Design

Array initialization, length & references • HAS-A • collaboration



From array memory → collaborating objects

CSC241 — Object-Oriented Programming • BAI Section • Fall 2026 • COMSATS University Islamabad, Lahore Campus
Instructor: Dr. Muhammad Shahid Bhatti

Bridge from Lecture 8

CONTINUITY

Lecture 8 — Discover

Identify candidate classes from requirements.

Assign responsibilities and collaborators.

Sketch an initial conceptual class diagram.

Lecture 9 — Organize + Connect

Store multiple references using object arrays.

Connect objects through fields and method calls.

Choose HAS-A composition when IS-A inheritance would be misleading.

Lecture 10 — Deepen

Multi-level composition.

Cohesion, coupling, ownership, and lifetime dependency.

Association, aggregation, composition, and cloning.

Today: create Java arrays, trace array references, then connect objects through composition.

Lecture 9 scope from the course plan

CDF ALIGNMENT

1 Object arrays + HAS-A

Arrays hold values or references; objects hold reference fields.

Use Student[] inside a CourseSection to store references.

2 Passing & returning objects

Objects can be method arguments.

Methods can return references to objects.

3 Composition vs inheritance

HAS-A models collaboration/containment.

IS-A models specialization.

Choose based on domain meaning, not code reuse alone.

Formal association / aggregation / composition notation is intentionally deferred to Lecture 10–11.

Learning outcomes

OUTCOMES

Recognize

Explain array indexing, length, and HAS-A relationships.

Implement

Initialize object arrays and reference fields using constructors.

Pass

Pass an object reference to a method and predict what the method can change.

Return

Return object references while preserving encapsulation decisions.

Choose

Defend composition vs. inheritance for a given design.

Compile

Create and traverse arrays in a collaborating-object program.

Why arrays belong in this OOP lecture

We move from handling one object reference to organizing several references.

ONE OBJECT

```
Student first =
    new Student(...);
```

One variable → one object

MANY OBJECTS

```
Student[] students =
    new Student[3];
```

One array → three reference slots

COLLABORATION

CourseSection HAS-A Student[] roster

A class manages many references

Teaching question: Does new Student[3] create three Student objects?

LECTURE 09

Declaration → creation → initialization

SYNTAX

Each step has a different meaning in Java.

1 DECLARE

```
int[] marks;
// reference variable
```

2 CREATE

```
marks = new int[4];
// new heap array
```

3 SET VALUES

```
marks[0] = 78;
marks[1] = 85;
```

ALTERNATIVE: INLINE INITIALIZER

```
int[] scores = {78, 85, 91, 67};
```

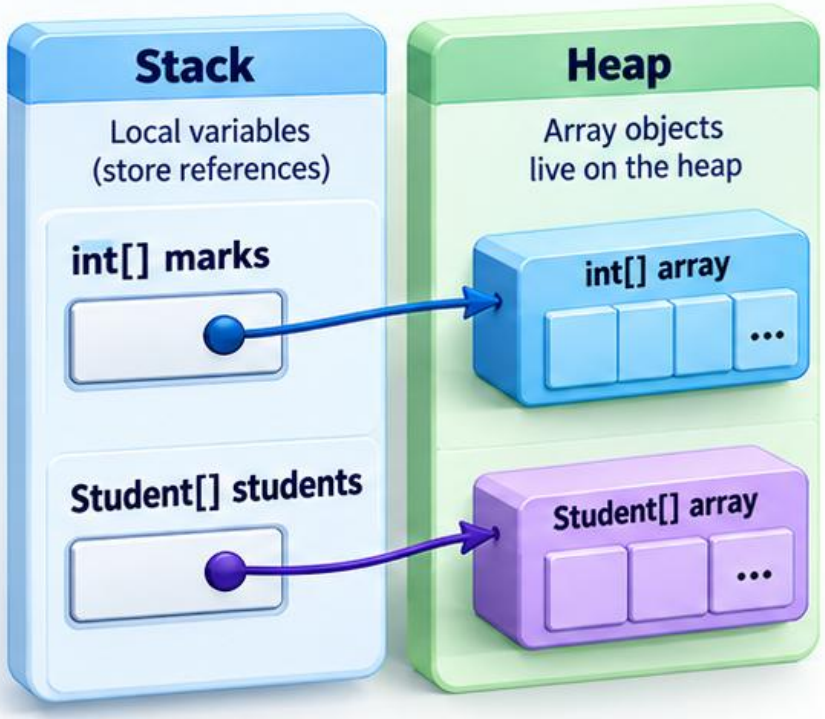
Rule: the array reference can be reassigned, but an existing array's length cannot change.

3D View of Java Arrays in Memory

references, length, and default initialization

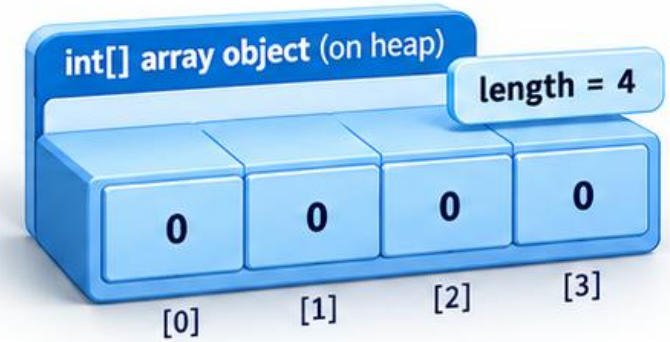
1 Array variable stores a reference

An array variable holds a reference to an array object on the heap, not the array contents.



2 Primitive array initialization

```
int[] marks = new int[4];
```

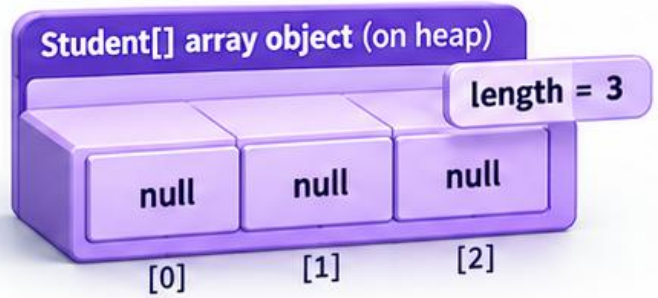


Default values (for primitive arrays)

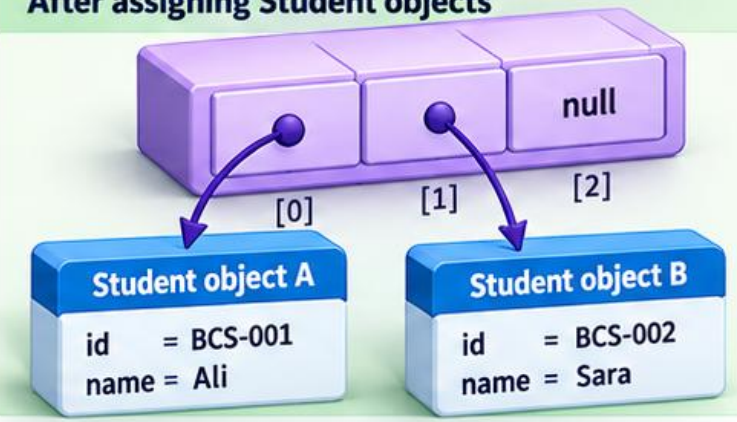
`int`	→	0
`double`	→	0.0
`boolean`	→	false
`char`	→	`'\u0000'` (null character)

3 Object array initialization

```
Student[] students = new Student[3];
```



After assigning Student objects



Key ideas

1. Arrays are fixed-size objects.
2. The variable holds a reference.
3. **length** is the array size.
4. Primitive arrays store actual values.
5. Object arrays store references.
6. New object arrays start with **null** elements.

Important

`new Student[3]` creates the array object, **not** 3 Student objects.

The length field: capacity versus filled elements

Access an array's length using marks.length — not marks.length().

JAVA

```
int[] marks = new int[5];
marks[0] = 78;
marks[1] = 85;
System.out.println(marks.length);
// prints 5
```

Array object: length = 5



marks.length = 5, although only 2 elements have been explicitly assigned.

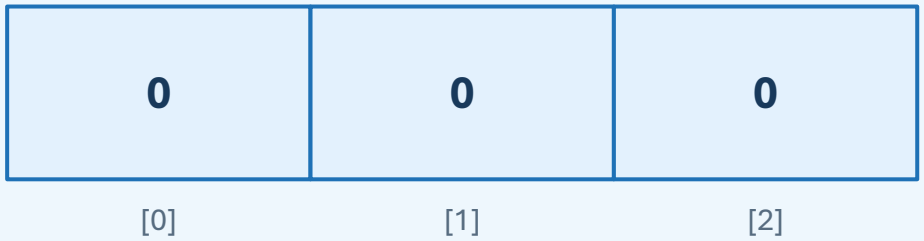
With object arrays, a separate count variable can track how many positions are in use.

Primitive arrays versus object arrays

MENTAL MODEL

Both arrays are heap objects; their indexed slots hold different kinds of data.

```
int[] marks = new int[3];
```



Elements hold primitive int values.

```
Student[] people = new Student[3];
```



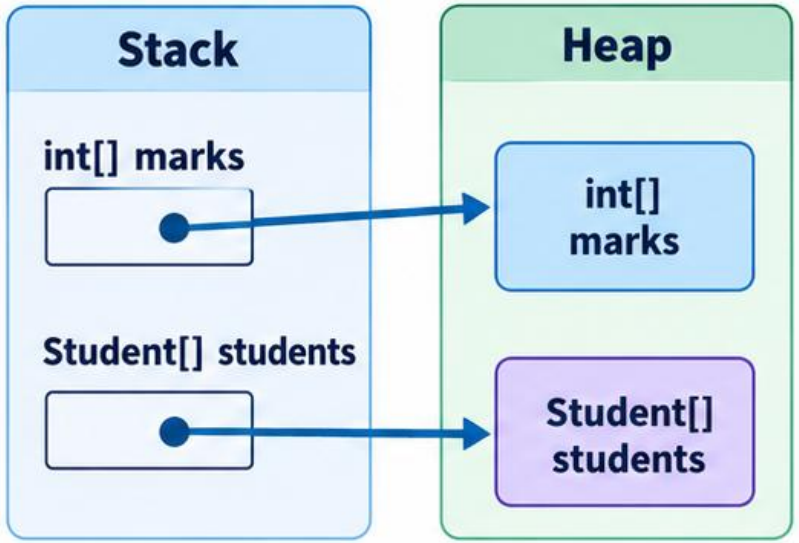
Elements hold Student references.

new Student[3] does not invoke the Student constructor three times.

How Arrays Are Managed in Memory (Java)

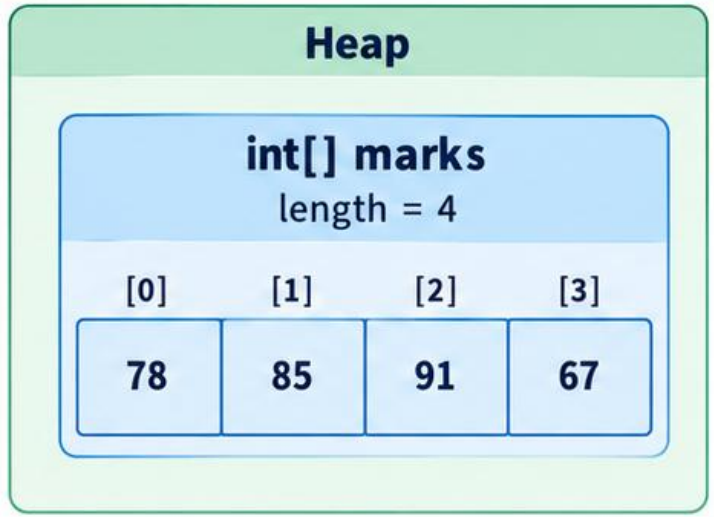
Primitive arrays vs object arrays

1 Array variable stores a reference



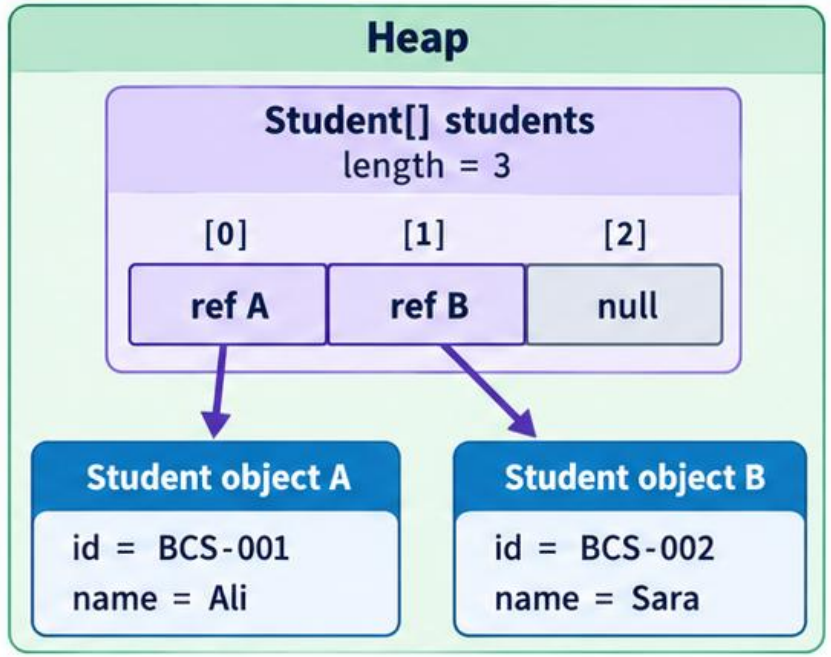
The variable itself holds a reference, not the whole array contents.

2 Primitive array in memory



- **Primitive arrays store the actual values directly in the array slots.**
- Indexes: 0 to length - 1.
- Array size is fixed after creation.

3 Object array in memory



- **An object array stores references.** The actual objects live separately on the heap.
- **new Student[3]** creates the array, not the Student objects.

Key Ideas

1. **Arrays are fixed-size objects.**
2. Array elements are stored in indexed slots.
3. **Primitive arrays** store values.
4. **Object arrays** store references.
5. Unfilled object slots default to null.

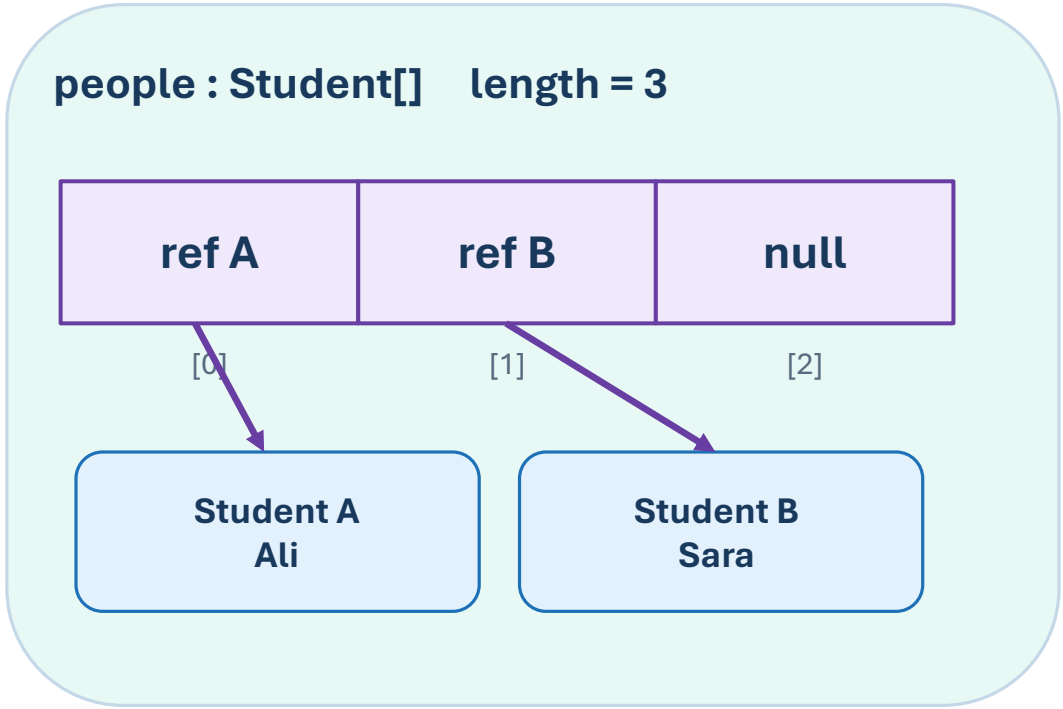
Initialize an object array: two separate operations

Create the array object first, then create and assign the Student objects.

CODE

```
Student[] people = new Student[3];
// [null, null, null]

people[0] = new Student("BAI-001", "Ali");
people[1] = new Student("BAI-002", "Sara");
// people[2] remains null
```



Important: people[2].getName() throws NullPointerException until that element refers to a Student.

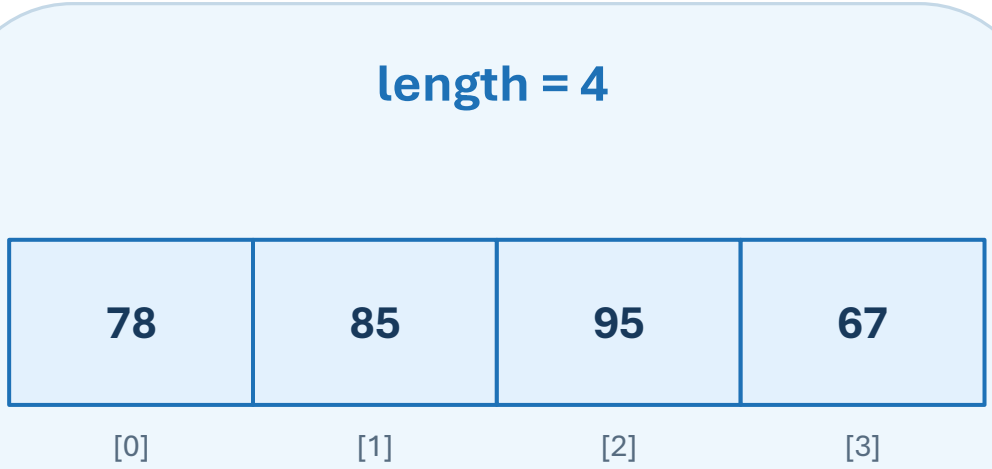
Indexed access, updates and bounds

The valid indexes of an array with length n are 0 through n - 1.

EXAMPLE

```
int[] marks = {78, 85, 91, 67};
System.out.println(marks[0]);  // 78
marks[2] = 95;                 // update
System.out.println(marks.length); // 4

// marks[4] --> runtime exception
```



Ask students: Do we get a compile-time error or ArrayIndexOutOfBoundsException?

Looping through arrays without null mistakes

TRAVERSAL

An enhanced for loop iterates values/references; an index loop also provides positions.

INDEX LOOP

```
for (int i = 0; i < people.length; i++)
{
  if (people[i] != null) {

System.out.println(people[i].getName())
;
  }
}
```

ENHANCED FOR

```
for (Student student : people) {
  if (student != null) {

System.out.println(student.getName());
  }
}
```

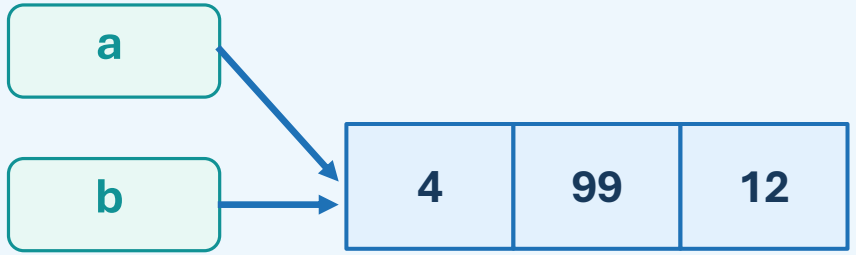
Both loops visit 3 slots; the null test prevents dereferencing an empty slot.

Reference sharing: assigning an array variable

Assignment copies the reference value. No second array object is created.

PREDICT THE OUTPUT

```
int[] a = {4, 8, 12};
int[] b = a;
b[1] = 99;
System.out.println(a[1]); // 99
System.out.println(a == b); // true
```



ONE array object, TWO references

Compare later with cloning/copying: an array alias is not a shallow or deep copy.

A HAS-A relationship with an array field

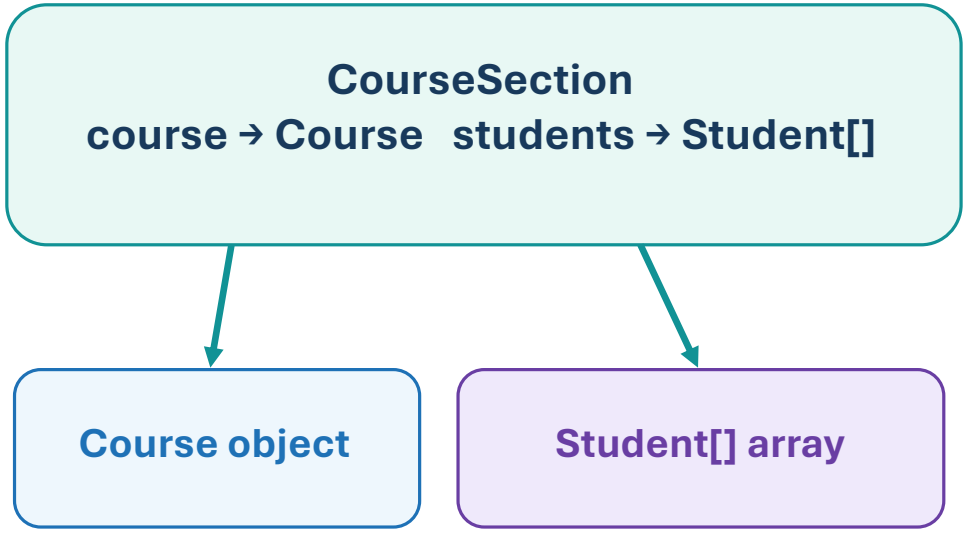
BRIDGE TO COMPOSITION

A CourseSection holds one Course reference and one Student[] reference.

CLASS DESIGN

```
class CourseSection {
  private Course course;
  private Student[] students;
  private int count;

  CourseSection(Course c, int capacity) {
    course = c;
    students = new Student[capacity];
  }
}
```



The Student objects can then be referenced by individual array slots.

Live coding challenge — a fixed-capacity roster

LIVE CODING

Use arrays, constructors and a class method; do not use ArrayList.

COURSESECTION METHOD

```
boolean enroll(Student student) {
  if (student == null ||
      count == students.length)
    return false;
  students[count] = student;
  count++;
  return true;
}
```

CAPACITY

students.length = total array slots

COUNT

count = how many students have enrolled

Extend it: print only indexes 0 to count - 1.

Prediction checkpoint — trace the array state

THINK → PAIR → EXPLAIN

Draw the array after each line before running the code.

CODE TRACING

```
Student[] a = new Student[2];
Student s = new Student("BAI-001", "Ali");
a[0] = s;
a[1] = a[0];
Student[] b = a;
b[0].setName("Sara");

// What does a[1].getName() print?
```

ANSWER THESE FIRST

1. How many array objects?
2. How many Student objects?
3. Do a[0] and a[1] refer to the same object?
4. Does b refer to another array?

After discussion: justify your answers with a reference diagram, not just the printed output.

The big idea: classes rarely work alone

MOTIVATION

Real OO systems are graphs of collaborating objects, not isolated class definitions.



Each relationship should answer a domain question: “Why does this object need to know that object?”

What does “composition” mean in everyday OO discussion?

HAS-A

Core idea

A class contains a field whose type is another class.

The containing object can use that referenced object to perform work.

The relationship is read naturally as “A has a B”.

Example: CourseSection has a Course object.

JAVA

```
class CourseSection {
    private Course course;
    private int capacity;

    CourseSection(Course course, int capacity) {
        this.course = course;
        this.capacity = capacity;
    }
}
```

Today we use “composition” broadly as object composition / HAS-A. Strong lifetime semantics come in Lecture 10.

Why use object composition?

DESIGN BENEFIT

Model the domain

A CourseSection should refer to the Course it offers.

A Registration may connect a Student and CourseSection.

Reuse behavior

The containing object can delegate to the component instead of copying its data or logic.

One Course object can serve multiple sections.

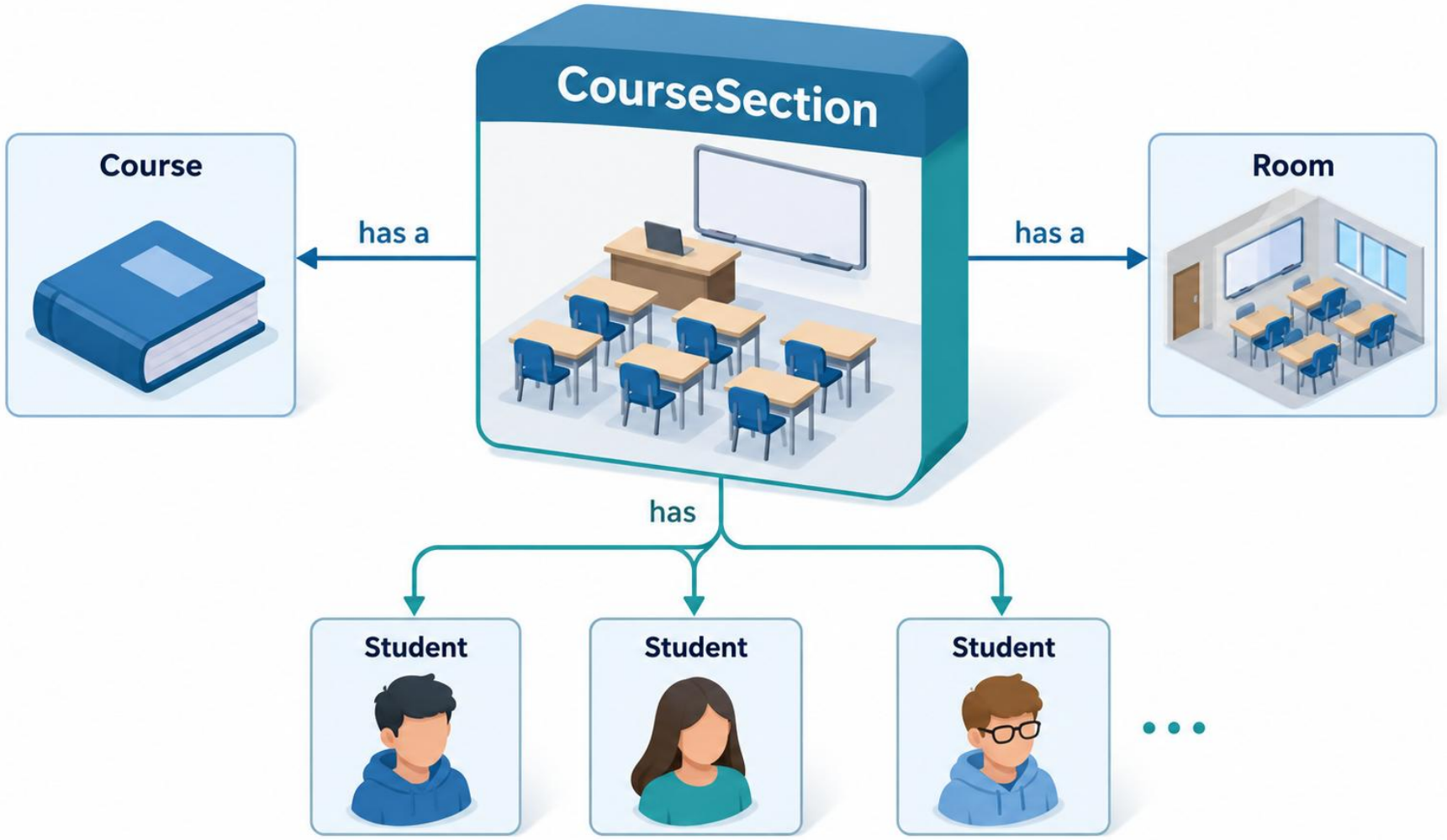
Localize change

If Course changes how it formats its title, CourseSection can keep using course.getDisplayName().

Fewer duplicated fields means fewer consistency bugs.

Running example — CourseSection has related objects

VISUAL MODEL



Reference fields connect objects

COURSE.JAVA

```
class Course {
    private String code;
    private String title;

    Course(String code, String title) {
        this.code = code;
        this.title = title;
    }

    String getCode() { return code; }
}
```

COURSESECTION.JAVA

```
class CourseSection {
  private Course course;
  private String section;
  private Student[] students;
  private int count;

  CourseSection(Course c, String s, int cap
    course = c;
    section = s;
    students = new Student[cap];
  }
}
```

Live coding checkpoint 1 — build the two classes

LIVE CODING

Predict → Code

Create Course.java, Student.java, then CourseSection.java.

CourseSection uses a Course and a Student[] array.

Ask students: does CourseSection contain a copy of Course or a reference to the same object?

Compile before adding extra methods.

TERMINAL

```
javac Student.java Course.java CourseSection.java

// No main method yet.
// Successful compilation means the type relationship
```

Teaching move: intentionally misspell Course once and use the compiler error to discuss type references.

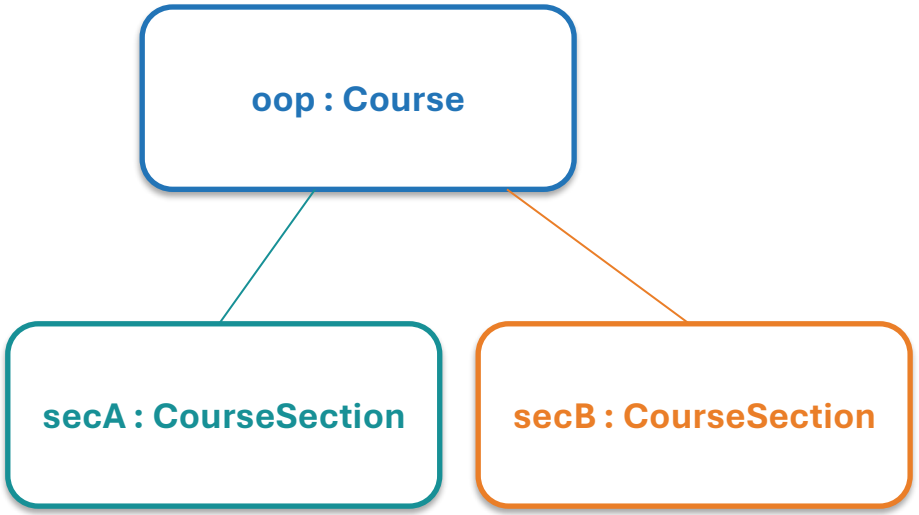
Create objects, then wire them together

MAIN.JAVA

```
Course oop = new Course(
    "CSC241",
    "Object-Oriented Programming"
);

CourseSection secA = new CourseSection(oop, "A", 40);
CourseSection secB = new CourseSection(oop, "B", 40);
```

OBJECT GRAPH



Two sections can reference the same Course object.

Delegate instead of duplicating data

Weak design — duplicate course data

CourseSection stores courseCode and courseTitle itself.

If title changes, every section copy must be updated.

State can become inconsistent.

BETTER

```
String getLabel() {
    return course.getCode() + "-" + section;
}

// CourseSection asks Course for course data.
// It does not duplicate Course's internal state.
```

Composition often reduces duplicate state by letting objects ask collaborators for what they need.

Constructor injection makes required relationships explicit

JAVA

```
class CourseSection {
  private final Course course;
  private final String section;
  private final Student[] students;

  CourseSection(Course c, String s, int cap) {
    if (c == null || cap < 0)
      throw new IllegalArgumentException();
    course = c; section = s;
    students = new Student[cap];
  }
}
```

Why this is useful

A CourseSection requires a valid Course and array capacity.

The dependency is visible in the constructor signature.

final prevents the field from being reassigned later.

The constructor establishes the relationship at object creation.

Passing objects to methods

COURSESECTION.JAVA

```
class CourseSection { // relevant excerpt
  private Student[] students;
  private int count;

  boolean enroll(Student student) {
    if (student == null ||
        count >= students.length)
      return false;
    students[count++] = student;
    return true;
  }
}
```

What is passed?

The method receives a copy of the reference value.

Both caller and parameter can refer to the same Student object.

The method may call Student methods through that reference.

Java still uses pass-by-value.

Live example — pass a real Student object

LIVE CODE

MAIN.JAVA

```
Student a = new Student(
    "SP26-BAI-005",
    "Abeer Amina"
);

CourseSection secA = new CourseSection(oop, "A", 40);
boolean ok = secA.enroll(a);
System.out.println(ok);
```

Ask before running

Which object does parameter student refer to?

Does enroll() create another Student object?

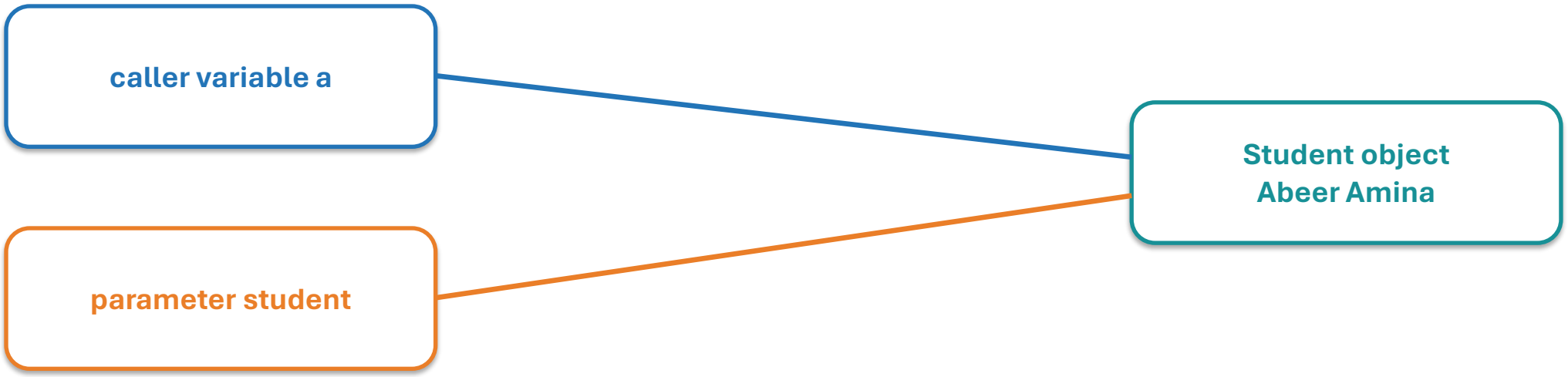
What changes if section capacity is already full?

What happens if a is null?

Java passes object references by value

IMPORTANT

The reference is copied; the object is not copied.



Two reference values → one object

Mutation vs. reassignment inside a method

PASS-BY-VALUE

```java
static void rename(Student s) {
    s.setName("Updated Name");
}

static void replace(Student s) {
    s = new Student("X", "Another Student");
}
```

rename(s)

Mutates the shared object.

Caller observes the changed name.

replace(s)

Only reassigns the local parameter variable.

Caller still holds its original reference.

Returning objects from methods

JAVA

```
class CourseSection {
    private Course course;

    Course getCourse() {
        return course;
    }
}

Course returned = secA.getCourse();
System.out.println(returned.getCode());
```

What is returned?

A reference value to an existing Course object.

No automatic object copy occurs.

The caller may now collaborate with that Course object.

Returning references is powerful—but affects encapsulation choices.

Return a collaborator — but decide what clients may do

Returning the object can be appropriate

Course is intended to be a public domain collaborator.

Clients need meaningful Course behavior.

Course protects its own invariants.

Returning internal mutable state can be risky

A client may obtain a reference and modify state you wanted controlled.

Do not automatically create getters for every object field.

Expose behavior or read-only information according to the class API.

Encapsulation still matters after objects are composed.

Required collaborator or null?

VALID STATE

JAVA

```
CourseSection(Course course, String section) {
    if (course == null)
        throw new IllegalArgumentException("course
    this.course = course;
    this.section = section;
}
```

Design question

Can a valid CourseSection exist without a Course?

If no, reject null at construction.

A relationship is part of the object invariant when the component is required.

Passing a composed object through methods

OBJECT FLOW

JAVA

```
static void printSection(CourseSection s) {
    Course c = s.getCourse();
    System.out.println(c.getCode());
}

printSection(secA);
```

Trace the references

main holds secA.

printSection receives a copy of secA's reference.

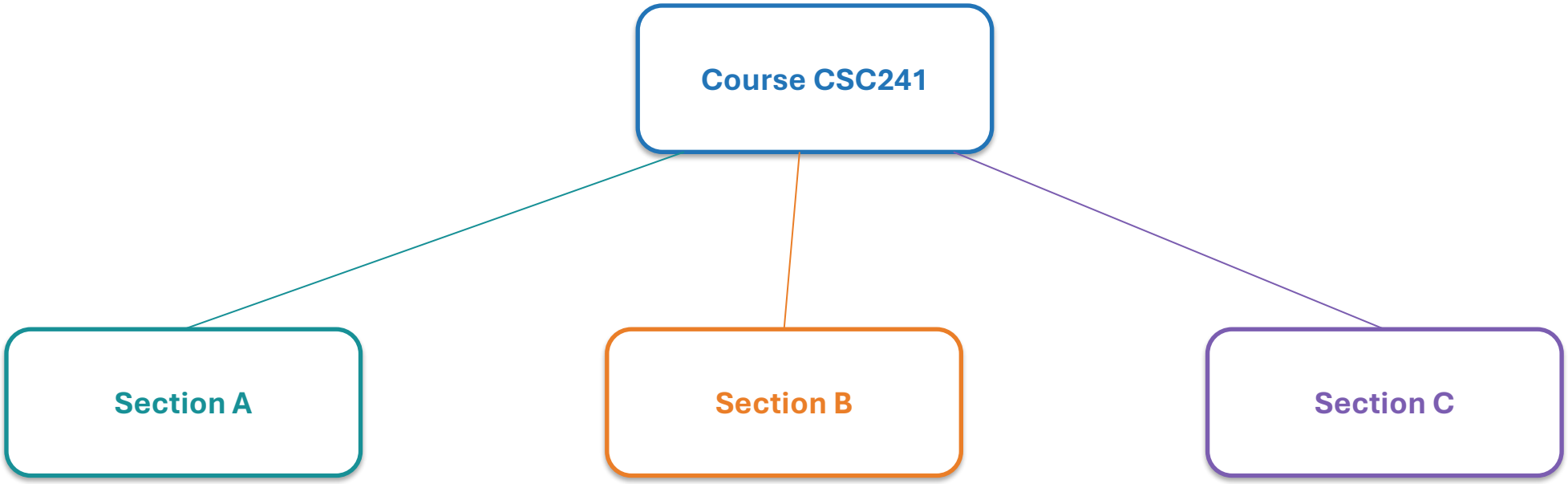
secA holds a Course reference.

getCourse() returns a copy of that Course reference.

No domain object is copied by these assignments.

One object can be shared by several owners of references

Sharing a collaborator is normal when the domain says the same object is being referenced.



Do not create three duplicate Course objects merely because there are three sections.

Passing an array to a method: what is copied?

PASS-BY-VALUE

Both the caller and parameter can reach the same array object.

REFERENCE COPY

```
static void change(int[] x) {
  x[0] = 42;
  x = new int[]{9, 9};
}

int[] a = {1, 2};
change(a);
System.out.println(a[0]); // 42
```

MUTATION

x[0] = 42 modifies the shared array.

REASSIGNMENT

x = new int[]{9, 9} changes only x, not a.

Java always passes arguments by value — arrays are not an exception.

Should a getter return the internal array?

An array reference returned by a getter gives callers access to its elements.

EXPOSED INTERNAL ARRAY

```
Student[] getStudents() {
  return students;
}

Student[] external = sec.getStudents();
external[0] = null;  // changes roster!
```

SAFER CLASS API

getStudentAt(index)
getEnrolledCount()
printRoster()

IF A COPY IS NEEDED

students.clone() copies the array slots but not the Student objects.

Later: compare reference aliasing, shallow array copies, and copying the objects themselves.

Composition vs. inheritance: two very different questions

DESIGN CHOICE

Inheritance asks: IS-A?

Is every subtype truly a specialized form of the supertype?

Example: Car is a Vehicle.

Inheritance expresses substitutability and shared type behavior.

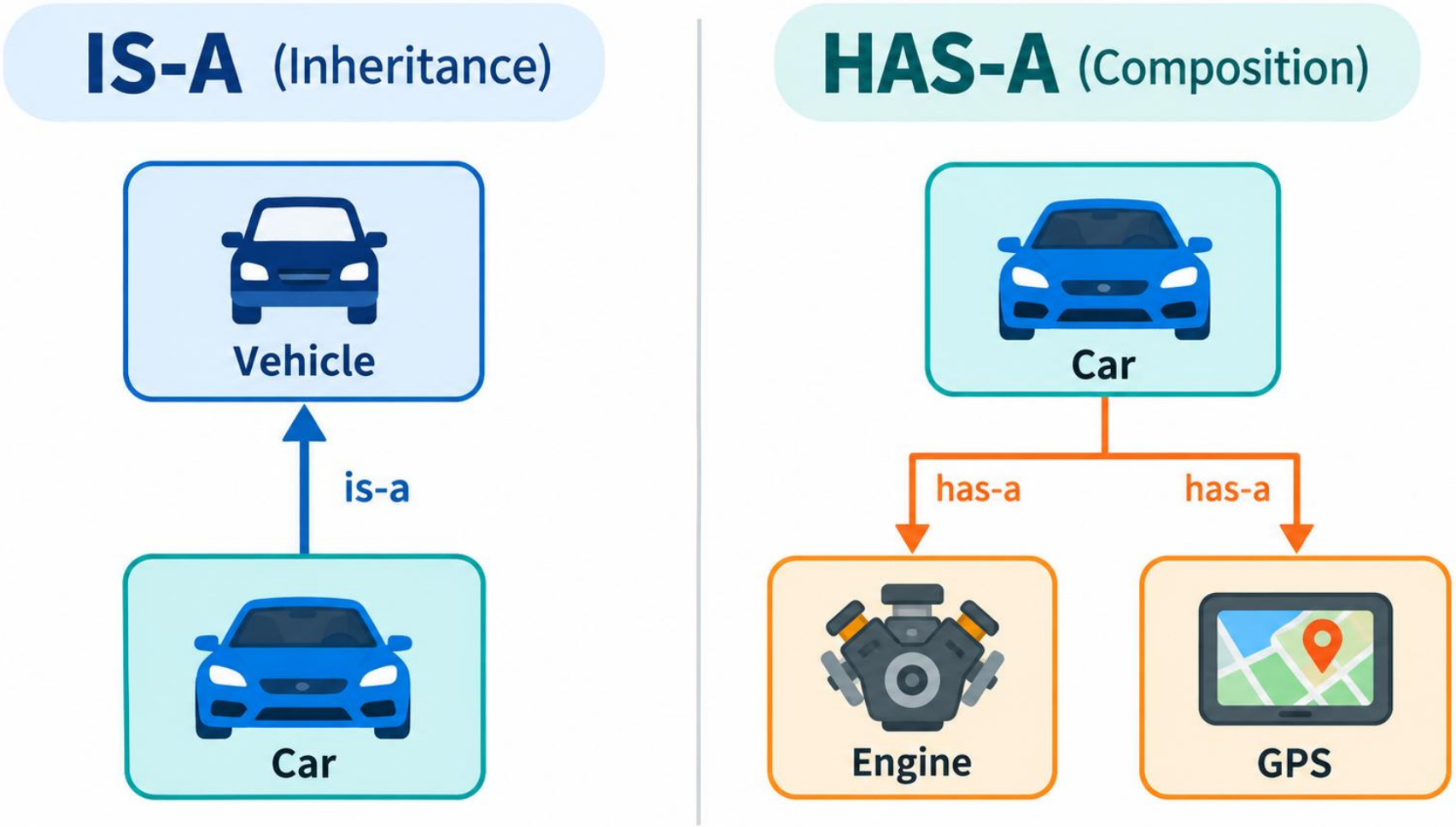
Composition asks: HAS-A / USES-A?

Does this object need another object to fulfill its responsibilities?

Example: Car has an Engine.

Composition expresses collaboration through object references.

Visual comparison — IS-A vs. HAS-A



A quick decision test

HEURISTIC

1. Can I naturally say “A is a B”?
2. Should an A be usable wherever a B is expected?
3. Or does A simply need / contain / collaborate with B?
4. Am I considering inheritance only to reuse code?

If the answer is “has/uses” rather than “is,” composition is usually the clearer model.

Bad inheritance example — CourseSection extends Course?

DESIGN SMELL

BAD IDEA

```
// Misleading model
class CourseSection extends Course {
    private String section;
}
```

Why it is wrong

A CourseSection is not a Course; it is an offering of a Course.

Substitution makes little domain sense.

A Course may have many sections.

Correct direction: CourseSection has a Course reference.

Bad inheritance example — Student extends Address?

DESIGN SMELL

Wrong reasoning

“Student needs address fields, so inherit from Address.”

This confuses code reuse with domain meaning.

A Student is not an Address.

BETTER

```
class Student {
    private Address address;   // HAS-A

    Student(Address address) {
        this.address = address;
    }
}
```

Composition gives flexible collaboration

DESIGN BENEFIT

Swap collaborators

A class can receive a different component object through its constructor.

The containing class does not need to change its inheritance hierarchy.

Limit assumptions

The class depends only on what it actually uses from the collaborator.

This often keeps relationships closer to domain meaning.

Easier evolution

New components can be introduced without claiming a new IS-A relationship.

Formal interface-based flexibility comes later in the course.

Refactor a data-heavy design into object composition

REFACTORING

BEFORE

```
// BEFORE: duplicate primitive data
class Enrollment {
    String studentId;
    String studentName;
    String courseCode;
    String courseTitle;
}
```

AFTER

```
// AFTER: collaborate with domain objects
class Enrollment {
    private Student student;
    private CourseSection section;

    Enrollment(Student student, CourseSection section)
        this.student = student;
        this.section = section;
    }
}
```

Object composition can replace duplicated primitive state with meaningful object references.

Live coding checkpoint 2 — Enrollment has Student + CourseSection

LIVE CODING

Build in class

Create Enrollment.java with two final reference fields.

Add a constructor that requires Student and CourseSection.

Add a method display() that asks both collaborators for information.

Compile after each small step.

START HERE

```
class Enrollment {
    private final Student student;
    private final CourseSection section;

    Enrollment(Student student, CourseSection section) {
        this.student = student;
        this.section = section;
    }
}
```

Compile and run the collaborating-object example

TERMINAL

POWERSHELL / TERMINAL

```
cd CSC241_L09_BAI

javac *.java
java Main
```

Predict before execution

How many objects are created in Main?

Which object holds a reference to which?

Which references point to the same Course object?

What compiler error appears if CourseSection.java is omitted from compilation?

Trace the object graph — not just the output

PREDICT

MAIN.JAVA

```
Student s = new Student("SP26-BAI-001", "Abeer Amina");
Course c = new Course("CSC241", "OOP");
CourseSection sec = new CourseSection(c, "A", 40);
Enrollment e = new Enrollment(s, sec);

Course again = sec.getCourse();
```

Questions

How many Course objects exist?

Does again == c evaluate to true?

Does Enrollment contain a copy of Student?

Which variables are references, and which objects live independently of those variable names?

Object composition and encapsulation work together

Keep fields private

Reference fields are still internal state.

Do not expose them merely because they are objects.

Expose meaningful behavior

Prefer sec.getCourseCode() or sec.getLabel() if clients only need information.

Return the Course object only when collaboration is appropriate.

Protect invariants

Reject invalid null collaborators when the relationship is mandatory.

Avoid duplicate state that can disagree.

Common mistakes with object composition

MISCONCEPTIONS

Duplicate object data

Copy collaborator fields instead of keeping a reference.

Public reference fields

Clients can replace relationships without validation.

Confuse reference copy with object copy

Assignment copies the reference value, not the object.

Use inheritance for convenience

Code reuse alone is not an IS-A relationship.

Ignore null invariants

Required collaborators should not silently become null.

Return internals automatically

Decide API exposure deliberately.

In-class design activity — Library borrowing

TEAM ACTIVITY

A Loan connects one Member with one BookCopy. A Librarian creates the Loan after checking availability.

Task A — HAS-A

Which reference fields should Loan contain?

Which object should own dueDate?

Task B — method flow

Design borrow(Member m, BookCopy copy).

Which object(s) are passed to the method?

Task C — inheritance test

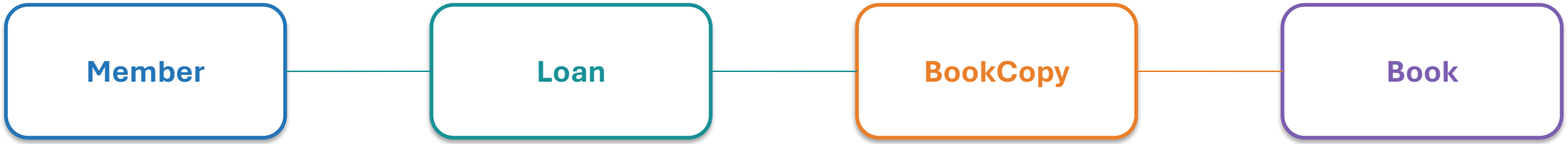
Should Loan extend Member? Why not?

Should BookCopy extend Book? Discuss carefully.

Deliverable: a 4-class sketch + one constructor + one method signature.

Activity debrief — one reasonable design

MODEL ANSWER



Loan

has Member, BookCopy, dueDate

Borrowing service

accepts Member + BookCopy arguments

Inheritance check

Loan is not Member; HAS-A is the right relationship.

Lecture recap & exit ticket

RECAP

Remember

Arrays have fixed length; object arrays initially store null references.

Object arrays and HAS-A fields store references, not object copies.

Mutation through a shared reference is visible; parameter reassignment is local.

Composition and inheritance express different domain meanings.

Prefer composition when the relationship is “has/uses,” not “is.”

Exit ticket

Why does new Student[3] initially contain only nulls?

Why is students.length a field rather than a method call?

Give one case where returning an internal object may weaken encapsulation.

Why is CourseSection extends Course a poor model?

Draw the object graph for Student → Enrollment → CourseSection → Course.

Next: Lecture 10 — multi-level composition, ownership/lifetime, cohesion/coupling, and stronger relationship semantics.